

Homework 2: Convex Optimization Problems

Instructions: Submit a single Jupyter notebook (.ipynb) of your work to Canvas by 11:59pm on the due date. All math must be formatted using LaTeX. **Be sure to show all the work involved in deriving your answers! If you just give a final answer without explanation, you may not receive credit for that question.**

You may discuss the concepts with your classmates, look on the internet for help, or ask an LLM for advice, but first try hard to solve problems on your own, and always write up the answers entirely on your own. Make sure you understand the concepts and solutions—don't just look up the answers or use AI to answer it for you. **Cite any sources you used outside of the class material (webpages, LLMs, etc.).**

1. Exercise 4.1 in B&V. Instead of hand drawing the feasible set, plot it using Python (you may find `matplotlib.patches.Polygon` useful for doing this.)
2. **Linear Programming.** Consider you need to ship n products from m warehouses to n destinations. Each warehouse can ship to at most one product to a single destination (we can assume $m \geq n$.) The distance from the i th warehouse to the j th destination is given as d_{ij} . You want to ship all n products with minimal total distance traveled. Set this problem up as a linear programming problem, with $x_{ij} = 1$ if the i th factory ships to the j th destination, and $x_{ij} = 0$ otherwise. However, to make this a linear programming problem, rather than a discrete optimization, set up the optimization variables x_{ij} to be continuous and constrained within $[0, 1]$.
 - (a) Write down a formula for the objective function $f(x)$ and the inequality / equality constraint functions. This should be in the standard form of a linear programming problem. **Note:** Even though the variable x is naturally in the form of an $m \times n$ matrix, you can think of it as a vector in $\mathbb{R}^{mn}$.
 - (b) Assuming all of the distances are strictly positive, i.e., $d_{ij} > 0$, give an argument for why the optimal solution will actually always be at integers, i.e., each $x_{ij}^* = 0$ or 1.
 - (c) Use CVXPY to solve the specific problem given by the following distances

$$D = [d_{ij}] = \begin{bmatrix} 150 & 37 & 88 \\ 23 & 42 & 16 \\ 105 & 34 & 7 \\ 200 & 95 & 12 \end{bmatrix}$$

- (d) Modify the problem for the case that each factory can ship an unlimited number of products. What are the new constraint functions? Give an analytic solution to this problem.
 - (e) Solve the modified problem using CVXPY for the same distances given in part (c), and verify that the answer matches your analytic solution.
3. Exercise 4.3 in B&V.
4. Use CVXPY to solve problem 3 above and verify the analytic solution.

5. Exercise 4.4 in B&V.

6. **Quadratic Programming** (Extended version of Exercise 4.22 in B&V) Consider the QCQP

$$\begin{aligned} & \text{minimize} && (1/2)x^T A x + b^T x + c \\ & \text{subject to} && x^T x \leq 1, \end{aligned}$$

with $A \in S_{++}^n$, $b \in \mathbb{R}^n$, and $c \in \mathbb{R}$.

- (a) Show that $x^* = -(A + \lambda I)^{-1}b$, where $\lambda = \max\{0, \bar{\lambda}\}$, and $\bar{\lambda}$ is the largest solution of the nonlinear equation

$$b^T(A + \lambda I)^{-2}b = 1.$$

- (b) Consider the relationship of this problem to the group invariance in the previous problem (Exercise 4.4). Under what conditions on the parameters A, b, c is this QCQP invariant to permutations of x ? **Hint:** Be sure to describe the most general conditions under which permutation invariance will hold. The trickiest is the condition on the matrix A - be sure to state the condition on A that ensures permutation invariance *and* symmetric positive-definiteness.
- (c) Using the result of Exercise 4.4, simplify the above QCQP to a one-dimensional optimization problem in the case that permutation invariance holds (that is, the conditions you found in part (b) hold.) Write expressions for the (univariate) objective function and (univariate) inequality constraint.
- (d) For dimension $n = 8$, pick an explicit set of parameters A, b, c that satisfy the permutation invariance conditions in part (b). You **may not** pick A to be diagonal, $b = 0$, or $c = 0$, but any other set of parameters are okay! (Just to be clear, $A = 0$ is considered a diagonal matrix and is thus not allowed.) Now, use CVXPY to solve this problem in the original (n -dimensional) form. Use CVXPY to solve it again in the reduced 1-dimensional form you derived in part (c) to confirm that the solution is the same.